

You are acting as a Senior SYNAPT Solution Architect, AI/Agent Architect, Data Engineer, Knowledge Graph Architect, Enterprise Architect, and Technical Presentation Mentor.

I am participating in a SYNAPT training/project exercise. I need to select one business use case and propose a complete, realistic, production-oriented solution using the concepts taught by the SYNAPT team.

The selected use case is:

INFRASTRUCTURE PROVENANCE

“Outages Trace to Untracked Infrastructure Changes”

The scenario is from the SYNAPT training material.

The scenario describes a telecom/network environment where routers, firmware versions, configuration baselines, routine patches, emergency hotfixes, planned maintenance, security changes, and vendor-applied changes are managed through different systems and teams.

When something goes wrong, the post-incident team struggles to determine:

- What actually changed on the affected asset
- Who made the change
- When the change happened
- Why the change was made
- Which firmware/configuration was involved
- Whether the change is related to the outage
- Whether similar assets are exposed to the same problem
- Whether a previous incident had the same pattern

The core problem is the lack of a connected lineage of infrastructure changes. Root-cause analysis becomes slow and frequently inconclusive, and the organization may end with a “probable cause” rather than a strongly evidence-supported conclusion.

The proposed solution must use SYNAPT to connect the fragmented enterprise knowledge and provide an agent-driven, governed investigation workflow.

===== IMPORTANT OBJECTIVE
=====

Do NOT create a shallow “LLM + RAG + chatbot” solution.

I need to understand the complete solution from:

WHY → WHAT → HOW → DATA → INGESTION → KNOWLEDGE/PROCEDURAL STORAGE → GRAPH → RETRIEVAL → MCP → AGENT → AGENT REGISTRY → POLICY → DECISION → HUMAN APPROVAL → ACTION/RECOMMENDATION → EXPLAINABILITY → OBSERVABILITY → AUDIT → FEEDBACK

The final solution must be something I can confidently present to the SYNAPT team and defend during detailed technical Q&A.

I need to understand every component and why it exists.

Do not skip steps.

===== 1. START WITH THE BUSINESS PROBLEM
=====

First explain the problem in business terms before discussing technology.

Explain:

- What exactly is the infrastructure provenance problem?
- Why does it happen?
- Why can every individual system work correctly while the overall RCA process still fails?
- Which systems contain the fragmented information?
- Which teams own the information?
- Why does manual investigation take so long?
- Why is it difficult to reconstruct the complete timeline?
- Why are previous RCA findings not reused effectively?
- How does this affect MTTR?
- How does it cause repeated incidents?
- How does it create operational risk?
- How does it affect similar assets?
- What would the ideal future-state process look like?

Clearly show:

CURRENT STATE

versus

PROPOSED STATE

Do not introduce technology before explaining the problem.

===== 2. DEFINE ONE CONCRETE
DEMONSTRATION SCENARIO =====

Create one realistic end-to-end incident for the demonstration.

Example:

A core router experiences an outage.

The incident system reports:

- Asset ID

- Incident ID
- Time
- Symptoms
- Severity
- Impact

The agent must investigate the incident.

It should discover:

- affected asset
- current firmware
- previous firmware
- recent configuration changes
- recent change requests
- maintenance activity
- security patches
- vendor activity
- engineer/team responsible
- timestamps
- previous incidents involving the asset
- historical RCA findings
- similar assets
- relevant SOP
- relevant policy

The agent should construct an evidence-supported root-cause hypothesis.

Then it should determine the next appropriate action.

If the next action involves modifying infrastructure, determine whether human approval is required.

The complete workflow should be:

Incident → Investigation → Evidence gathering → Graph correlation → Historical comparison → Procedure retrieval → Policy evaluation → Root-cause hypothesis → Recommendation → Human approval if required → Action if approved → Validation → Audit → Feedback

===== 3. DATA DESIGN — VERY DETAILED
=====

This is one of the most important sections.

Do NOT use only three or four records.

Design a realistic demonstration dataset large enough to show relationships.

Target approximately:

- 10–20 infrastructure assets

- Multiple router models
- Multiple firmware versions
- Multiple configuration versions
- Multiple change records
- Multiple maintenance records
- Multiple security patches
- Multiple vendor activities
- Multiple incidents
- Multiple historical RCA records
- Multiple engineers/teams
- Several services
- Multiple locations/network segments
- Several SOPs
- Several policies
- At least one recurring historical failure pattern
- Several similar assets

The data must be internally consistent.

Every important entity should be connected through IDs.

For every dataset explain:

1. Why we need it
2. Source system
3. Example fields
4. Example records
5. Format
6. Whether it belongs in Knowledge Store or Procedural Store
7. Which graph entities it creates
8. Which graph relationships it creates
9. How the agent retrieves it
10. What happens if the data is stale
11. What happens if the data is missing
12. What happens if records conflict

===== 4. DATA CATEGORIES
=====

Design detailed data for the following categories.

A. INFRASTRUCTURE / ASSET DATA

Fields should include:

- asset_id
- hostname
- device_type
- vendor
- model
- serial/reference identifier

- location
- network_segment
- environment
- criticality
- owner_team
- operational_status
- dependencies
- current_firmware
- current_configuration

Explain why each field matters.

B. FIRMWARE DATA

Fields should include:

- firmware_id
- firmware_version
- vendor
- model
- release_date
- supported_devices
- known_issues
- vulnerabilities
- compatibility
- lifecycle_status

Explain how firmware history can be correlated with incidents.

C. CONFIGURATION BASELINE DATA

Fields should include:

- configuration_id
- asset_id
- baseline_version
- configuration_parameters
- approved_baseline
- effective_from
- effective_to
- previous_configuration
- approval_reference

Explain how configuration drift can be detected conceptually.

D. CHANGE MANAGEMENT DATA

Fields should include:

- change_id
- asset_id
- change_type
- requested_by
- approved_by
- executed_by
- request_timestamp
- approval_timestamp
- execution_timestamp
- implementation_window
- reason
- status
- before_state
- after_state
- change_description

Explain how change timing is correlated with incidents.

E. MAINTENANCE DATA

Fields should include:

- maintenance_id
 - asset_ids
 - maintenance_window
 - maintenance_team
 - purpose
 - planned_changes
 - actual_changes
 - start_time
 - end_time
 - status
-

F. SECURITY / PATCH DATA

Fields should include:

- patch_id
- vulnerability_id
- affected_assets
- patch_version
- deployment_date
- deployment_team
- urgency
- status
- remediation_reference

G. VENDOR ACTIVITY DATA

Fields should include:

- vendor_ticket_id
- asset_id
- vendor
- technician
- activity
- timestamp
- firmware_change
- configuration_change
- reason
- completion_status

Explain why vendor activity is important for provenance.

H. INCIDENT DATA

Fields should include:

- incident_id
- affected_asset_id
- start_time
- detection_time
- end_time
- symptoms
- severity
- impacted_service
- alerts
- resolution
- RCA_status
- incident_description

I. HISTORICAL RCA DATA

Fields should include:

- rca_id
- incident_id
- root_cause
- evidence
- affected_asset
- affected_firmware
- affected_configuration
- remediation
- validation_result

- similar_assets
- lessons_learned
- RCA_timestamp

Explain why historical RCA is valuable for future investigations.

J. SOP / PROCEDURAL DATA

Create realistic step-by-step procedures.

Example:

INCIDENT INVESTIGATION SOP

Step 1: Identify affected asset.

Step 2: Retrieve recent changes.

Step 3: Check firmware history.

Step 4: Compare configuration baseline.

Step 5: Check maintenance activity.

Step 6: Check security patch activity.

Step 7: Check vendor activity.

Step 8: Search historical incidents.

Step 9: Compare historical RCA findings.

Step 10: Identify connected evidence.

Step 11: Determine probable root cause.

Step 12: Identify similar assets.

Step 13: Generate remediation recommendation.

Step 14: Evaluate applicable policy.

Step 15: Request human approval if required.

Step 16: Execute approved remediation.

Step 17: Validate recovery.

Step 18: Record outcome.

Step 19: Provide feedback.

Do NOT simply state that this is a procedure.

Explain how the procedure is represented and retrieved through SYNAPT.

K. POLICY / GOVERNANCE DATA

Create policies such as:

- Critical infrastructure cannot be automatically modified.
- Firmware rollback requires an approved target version.
- Emergency changes require authorized approval.
- Changes outside approved maintenance windows require escalation.
- High-impact assets require human approval.
- Remediation cannot proceed without an applicable documented procedure.
- Evidence must sufficiently support the recommendation before action.

Explain how these policies participate in the workflow.

===== 5. KNOWLEDGE STORE VS PROCEDURAL
STORE =====

Explain this extremely clearly.

Create a table containing:

DATA TYPE → STORE → WHY → HOW USED BY AGENT

Examples:

Incident facts → Knowledge Store

Historical RCA → Knowledge Store

Firmware documentation → Knowledge Store

Configuration history → Knowledge Store

Change records → Knowledge Store

Infrastructure relationships → Graph/knowledge representation

Investigation SOP → Procedural Store

Remediation SOP → Procedural Store

Rollback procedure → Procedural Store

Explain why a procedure should not simply be treated as ordinary informational text.

Explain how Knowledge Store and Procedural Store work together.

===== 6. INGESTION ARCHITECTURE
=====

Explain the complete ingestion flow:

Source Systems → Adapters → Ingestion → Parsing → Normalization → Structuring → Knowledge/
Procedural processing → Graph construction → Indexing → SYNAPT stores

Explain every stage.

Answer:

- What does an adapter do?
- Why are adapters required?
- How do we deal with different source schemas?
- How do we normalize terminology?
- How do we normalize IDs?
- How do we normalize timestamps?
- How do we identify entities?
- How are relationships established?
- How are documents/chunks handled?
- How are procedures represented?
- How does graph construction fit into ingestion?
- How are updates handled?
- How are duplicate records handled?
- How are stale records handled?
- How are changed records handled?
- How are deleted records handled?
- How do we maintain source lineage?

IMPORTANT:

Do not invent proprietary SYNAPT internals.

Clearly distinguish:

CONFIRMED SYNAPT BEHAVIOR

from

PROPOSED APPLICATION-LEVEL DESIGN

from

IMPLEMENTATION ASSUMPTION

from

REQUIRES CONFIRMATION FROM SYNAPT TEAM

===== 7. GRAPH DATA MODEL =====

Design a detailed graph.

Potential entities:

Asset Router Firmware Configuration Change Maintenance Patch Vendor Engineer Team Incident RCA
Service Location Vulnerability

Potential relationships:

Asset → HAS_FIRMWARE → Firmware

Asset → HAS_CONFIGURATION → Configuration

Asset → AFFECTED_BY → Change

Change → EXECUTED_BY → Engineer

Change → APPROVED_BY → Engineer

Asset → INVOLVED_IN → Incident

Incident → HAS_RCA → RCA

RCA → IDENTIFIED_FIRMWARE → Firmware

Asset → DEPENDS_ON → Service

Asset → LOCATED_IN → Location

Asset → SAME_MODEL_AS → Asset

Explain every relationship.

Then demonstrate graph traversal for the incident.

Example:

Incident → Asset → Change → Firmware → Historical Incident → RCA → Similar Assets

Explain why this connected traversal is more valuable than looking at each record independently.

===== 8. SYNAPT RETRIEVAL TOOLS
=====

Use the exact SYNAPT concepts taught by the team.

Explain:

1. Get Stats
2. Trident Query
3. Trident Search
4. Trident Traverse
5. Get Chunks
6. Get Procedures
7. Feedback

For each explain:

- Purpose
- Input
- Output
- When the agent uses it
- Why it is useful
- Example in this project
- What happens after the tool returns

Use the training terminology accurately.

Explain:

GET STATS → Provides an overview of what exists in the provider/knowledge graph.

TRIDENT SEARCH → Finds relevant graph nodes for a query.

TRIDENT TRAVERSE → Starts from a graph node and follows relationships for the requested number of jumps.

GET CHUNKS → Performs direct semantic search against the Knowledge Store without relying on graph traversal.

GET PROCEDURES → Retrieves candidate procedures matching the requested task.

TRIDENT QUERY → General-purpose query interface that can orchestrate the other retrieval tools and provide a formatted response.

FEEDBACK → Allows the agent to report whether the retrieved information enabled successful task completion.

Do not claim every tool must always be called.

Explain:

WHEN TRIDENT QUERY IS ENOUGH

versus

WHEN EXPLICIT TOOLS ARE BETTER.

===== 9. AGENT ARCHITECTURE
=====

Design the complete agent.

Explain:

- Agent objective
- Input
- Context
- Tools
- Tool selection
- Investigation logic
- Evidence collection
- Graph traversal
- Procedure retrieval
- Policy evaluation
- Decision
- Recommendation
- Human approval
- Execution
- Validation
- Feedback

The agent workflow should be:

Incident → Agent → Understand task → Retrieve relevant evidence → Search graph → Traverse relationships → Retrieve relevant chunks → Retrieve applicable procedure → Evaluate policy → Correlate evidence → Generate root-cause hypothesis → Determine confidence/evidence sufficiency → Recommend action → Human approval if required → Execute approved action → Validate → Audit → Feedback

===== 10. WHAT THE LLM DOES VS DOES NOT
DO =====

This distinction is extremely important.

Clearly explain:

LLM:

- Understands the request

- Plans investigation
- Selects tools
- Interprets retrieved evidence
- Correlates evidence
- Produces a human-readable explanation
- Orchestrates the workflow

LLM IS NOT:

- The enterprise source of truth
- The authority for infrastructure facts
- Allowed to invent missing procedures
- Allowed to bypass policy
- Allowed to fabricate evidence
- Automatically allowed to modify critical infrastructure

SYNAPT:

- Provides governed enterprise knowledge/context
- Provides retrieval capabilities
- Provides graph-connected information
- Provides procedural information where available
- Receives feedback from agent task execution

===== 11. MCP
=====

Explain:

- What role MCP plays
- How the agent connects to SYNAPT tools
- What tools are exposed
- How tool calls occur
- Why standardized tool access is useful

Do not describe MCP as the entire governance layer.

Explain its relationship with Agent Registry.

===== 12. AGENT REGISTRY
=====

Explain Agent Registry as a governance/control layer.

Cover:

- Agent identity
- Allowed capabilities
- Allowed tools
- Allowed data
- Access restrictions
- Governance

- Controlled execution

Explain:

MCP vs. Agent Registry

Do not treat them as the same thing.

Show what this particular RCA agent is allowed to do.

Example:

ALLOWED:

- Read incident information
- Search infrastructure knowledge
- Traverse graph relationships
- Retrieve procedures
- Retrieve historical RCA
- Generate recommendations

RESTRICTED:

- Modify critical routers
- Execute firmware changes
- Change production configuration

Explain how permissions should be designed.

If exact SYNAPT Agent Registry implementation details are not known from the training material, explicitly state that they are an implementation detail requiring confirmation.

===== 13. POLICY AND PRECONDITION
HANDLING =====

Explain the difference between:

Knowledge Procedure Policy Decision

Example:

Knowledge: "Router R-104 currently runs firmware 7.2."

Procedure: "To investigate a router outage, check recent changes, firmware, configuration, historical incidents..."

Policy: "Critical routers cannot be modified without human approval."

Decision: "Firmware 7.2 is the leading root-cause hypothesis and rollback is recommended, subject to approval."

Workflow:

Agent → Retrieve policy → Evaluate preconditions → If satisfied → Continue

If not satisfied:

→ Do not perform restricted action → Inform user → Request human approval/escalation where appropriate

===== 14. EXPLAINABILITY — FIRST-CLASS REQUIREMENT =====

Explainability is mandatory.

Never produce a black-box answer such as:

“Firmware 7.2 caused the outage.”

Instead create an evidence chain.

Example:

Incident INC-2041 ↓ Affected Asset R-104 ↓ Firmware changed from 7.1 to 7.2 ↓ Change occurred two days before outage ↓ Historical Incident INC-1890 ↓ Same firmware 7.2 ↓ Similar symptoms ↓ Historical RCA identified firmware 7.2 ↓ 12 similar assets also run firmware 7.2

Then produce:

ROOT-CAUSE HYPOTHESIS: Firmware 7.2 is the leading evidence-supported hypothesis.

SUPPORTING EVIDENCE:

- Recent firmware change
- Temporal proximity
- Historical incident similarity
- Historical RCA
- Similar assets

ALTERNATIVE HYPOTHESES: List other plausible causes if relevant.

Explain:

- What evidence was used
- Where the evidence came from
- Which graph relationships were followed
- Timeline
- Historical evidence
- Confidence/evidence strength
- Alternative explanations
- Why the selected hypothesis is stronger

- What evidence is missing
- What the agent did NOT assume

Do not expose private chain-of-thought.

Provide evidence-based explanations, not hidden reasoning.

===== 15. CONFIDENCE / INSUFFICIENT
EVIDENCE =====

This is VERY IMPORTANT.

The system must NOT force an answer.

If the evidence is insufficient, the agent must explicitly communicate that it cannot establish a reliable conclusion.

Use messages such as:

"I am not confident in the root-cause determination because the available evidence is insufficient."

or:

"No sufficiently matching documentation or evidence was found in the available SYNAPT knowledge."

or:

"The retrieved records contain conflicting information, so a reliable root-cause conclusion cannot be established automatically."

The agent must NOT:

- Invent a root cause
- Invent a procedure
- Pretend evidence exists
- Treat an unsupported correlation as fact
- Execute an action based on insufficient evidence

When appropriate, the case should be escalated to the relevant human/network expert.

Clearly distinguish:

NO EVIDENCE

from

EVIDENCE SAYS NO

from

CONFLICTING EVIDENCE

from

SUFFICIENT EVIDENCE

===== 16. FALLBACK / ESCALATION BEHAVIOR =====

Do NOT design a generic fallback that invents an answer.

The safe behavior is:

Insufficient evidence → Explicit “not confident” message → No unsupported action → Escalate to human/expert when required

Examples:

CASE 1: No relevant evidence found.

Output: “I am not confident. No sufficiently matching documentation was found.”

Action: Escalate if investigation is operationally important.

CASE 2: Conflicting records.

Output: “I am not confident because the available records conflict.”

Action: Human investigation.

CASE 3: No applicable procedure.

Output: “No documented procedure matching this situation was found. I cannot safely invent the required steps.”

Action: Escalate.

CASE 4: Policy condition fails.

Output: “The recommended action is restricted by policy and requires human approval.”

Action: Human approval.

CASE 5: Evidence is weak.

Output: “Firmware X is a possible contributing factor, but the available evidence is insufficient to establish it as the root cause.”

Action: Do not automatically remediate.

The primary safety principle is:

NO SUPPORTED EVIDENCE → NO FABRICATED ANSWER → NO UNSUPPORTED ACTION.

===== 17. HUMAN-IN-THE-LOOP
=====

Classify actions into:

FULLY AUTOMATED

RECOMMENDATION ONLY

HUMAN APPROVAL REQUIRED

Example:

Read incident information → Automated

Search SYNAPT → Automated

Graph traversal → Automated

Historical RCA retrieval → Automated

Generate root-cause hypothesis → Automated

Generate remediation recommendation → Recommendation

Modify critical router → Human approval

Firmware rollback → Human approval

Production configuration change → Human approval

Explain why each category exists.

===== 18. OBSERVABILITY
=====

Observability is mandatory.

Design complete agent observability.

Track:

- Agent invocation
- Session ID

- Incident ID
- Request
- Agent identity
- Tools selected
- Tool calls
- Tool execution status
- Retrieval latency
- Number of nodes retrieved
- Number of chunks retrieved
- Traversal depth
- Procedures retrieved
- Policies evaluated
- Decision
- Recommendation
- Action attempted
- Human approval
- Execution status
- Errors
- Insufficient-evidence events
- Escalation events
- Final outcome
- Feedback

Show a sample execution trace:

Agent Started ↓ Incident received ↓ Get Stats ↓ Trident Search ↓ 8 relevant nodes ↓ Trident Traverse
 ↓ 3-hop traversal ↓ Get Chunks ↓ Historical RCA retrieved ↓ Get Procedures ↓ Investigation SOP
 retrieved ↓ Policy evaluated ↓ Evidence correlated ↓ Root-cause hypothesis generated ↓ Human
 approval required ↓ Recommendation generated ↓ Outcome recorded ↓ Feedback submitted

===== 19. EXPLAINABILITY VS OBSERVABILITY VS
 AUDIT =====

This distinction must be extremely clear.

Explainability answers:

“WHY did the agent reach this conclusion?”

Observability answers:

“WHAT did the agent do during execution?”

Audit answers:

“WHAT happened, WHEN, under WHICH identity, using WHICH evidence/action, and WHO approved it?”

Create a comparison table.

===== 20. AUDIT / TRACEABILITY
=====

Design how we can answer:

"What did the agent know?"

"What sources did it retrieve?"

"What tools did it call?"

"What graph nodes did it traverse?"

"What procedure did it retrieve?"

"What policy did it evaluate?"

"What decision did it make?"

"Why did it make that decision?"

"What action did it recommend?"

"Did it execute anything?"

"Who approved the action?"

"When did it happen?"

"What was the final result?"

"Was feedback recorded?"

===== 21. FEEDBACK / SELF-LEARNING
=====

Explain the SYNAPT feedback mechanism from the training.

Example:

Agent performs task → Task successful? → Yes/No → Feedback sent to SYNAPT → Feedback participates in improvement/self-learning loop

Explain carefully:

Feedback can indicate whether the information retrieved from SYNAPT enabled the agent to complete the task.

Do NOT claim that feedback automatically proves whether a piece of enterprise knowledge is factually correct.

Explain the distinction.

===== 22. SECURITY AND GOVERNANCE
=====

Cover:

- Least privilege
- Agent identity
- Tool access
- Data access
- Critical infrastructure protection
- Sensitive information
- Policy enforcement
- Human approval
- Auditability
- Unauthorized actions
- Access isolation
- Controlled execution

Clearly separate:

SYNAPT capability

from

PROPOSED APPLICATION-LEVEL SECURITY.

Do not invent proprietary SYNAPT functionality.

===== 23. COMPLETE END-TO-END
ARCHITECTURE =====

Create a professional architecture diagram.

Include:

DATA SOURCES ↓ Adapters ↓ Ingestion ↓ Normalization / Structuring ↓ Knowledge Store Procedural
Store Graph ↓ MCP ↓ Agent Registry ↓ RCA Agent ↓ Retrieval ↓ Evidence Correlation ↓ Policy
Evaluation ↓ Decision ↓ Recommendation / Human Approval ↓ Action ↓ Validation ↓ Feedback ↓
Observability ↓ Audit / Explainability

Show important connections between components.

Then explain EVERY box.

===== 24. END-TO-END SEQUENCE DIAGRAM
=====

Create a detailed sequence diagram for:

“Router outage investigation.”

Actors:

- Incident Management System
- RCA Agent
- Agent Registry
- MCP
- SYNAPT
- Trident Search
- Trident Traverse
- Knowledge Store
- Procedural Store
- Policy
- Human Approver
- Network/Change System

Show every important interaction.

===== 25. DEMO DATASET
=====

Create a realistic dataset.

Minimum recommendation:

Assets: 10–20

Firmware versions: 3–5

Configurations: 10+

Changes: 15+

Maintenance records: 5+

Security patches: 5+

Vendor activities: 5+

Incidents: 8–10

Historical RCAs: 5+

Engineers: 5+

Teams: 3-5

Services: 3-5

Locations/network segments: 3-5

SOPs: 3+

Policies: 5+

The dataset should contain:

- One strong root-cause pattern
- One similar-assets pattern
- One conflicting-evidence case
- One insufficient-evidence case
- One human-approval case

Ensure all IDs and relationships are internally consistent.

===== 26. ACTUAL SAMPLE DATA
=====

Provide realistic CSV/JSON-style sample records.

Do not simply write:

"Router data"

or

"Incident data."

Show actual fields and realistic values.

For example:

Asset:

asset_id: RTR-104

model: XR-9000

firmware: 7.2

location: Chennai-Core-DC

criticality: CRITICAL

Then show how:

RTR-104 → CHG-2041 → FW-7.2 → INC-3008 → RCA-1882

connect together.

Create enough records to demonstrate graph relationships.

===== 27. AGENT OUTPUT
=====

Design the final response displayed to the user.

Use a structure such as:

INCIDENT SUMMARY

Affected Asset

Incident Timeline

Recent Infrastructure Changes

Firmware History

Configuration History

Maintenance Activity

Vendor Activity

Historical Similar Incidents

Graph Relationships

Root-Cause Hypothesis

Supporting Evidence

Alternative Hypotheses

Evidence/Confidence Assessment

Policy Evaluation

Recommended Action

Human Approval Requirement

Similar Assets at Risk

Limitations / Missing Evidence

Audit / Trace ID

Do not expose private chain-of-thought.

===== 28. PRESENTATION
=====

Create a 10–15 minute presentation.

Suggested structure:

Slide 1: Problem

Slide 2: Current-State Fragmentation

Slide 3: Business Impact

Slide 4: Proposed SYNAPT Solution

Slide 5: Architecture

Slide 6: Data Sources and Ingestion

Slide 7: Knowledge Store vs Procedural Store

Slide 8: Graph Model

Slide 9: Agent + MCP + Agent Registry

Slide 10: Retrieval and Investigation Flow

Slide 11: Policy and Human-in-the-Loop

Slide 12: Explainability

Slide 13: Observability and Audit

Slide 14: Insufficient Evidence / Escalation

Slide 15: End-to-End Demo

Slide 16: Business Value

Slide 17: Future Enhancements

For every slide provide:

- Slide title
- Main message
- Diagram/content
- Exact points to show
- What I should say
- Likely question from SYNAPT team

===== 29. DEMO SCRIPT
=====

Create a live demo script.

Start with:

"An outage has occurred on Router R-104."

Then demonstrate:

1. Incident received
2. Agent identifies affected asset
3. Agent retrieves relevant information
4. Agent searches graph
5. Agent traverses relationships
6. Agent retrieves historical RCA
7. Agent retrieves investigation procedure
8. Agent evaluates policy
9. Agent correlates timeline
10. Agent generates evidence-supported hypothesis
11. Agent identifies similar assets
12. Agent determines whether remediation requires approval
13. Agent produces recommendation
14. Human approves if required
15. Action is executed if appropriate
16. Outcome is validated
17. Audit/trace is shown
18. Feedback is submitted

Also demonstrate at least one:

"Not confident / insufficient evidence"

scenario.

===== 30. TECHNICAL Q&A
=====

Generate difficult questions the SYNAPT team may ask.

Especially answer:

- Why SYNAPT?
- Why not just use an LLM?
- Why not just use a vector database?
- Why do we need a graph?
- Why Trident Search?
- Why Trident Traverse?
- Why Get Chunks?
- Why Get Procedures?
- When is Trident Query enough?
- Why use explicit tools?
- What happens during ingestion?
- What do adapters do?
- What goes into Knowledge Store?
- What goes into Procedural Store?
- How are relationships created?
- How does the agent select tools?
- What is MCP?
- What is Agent Registry?
- How is Agent Registry different from MCP?
- How do you restrict agent access?
- How do you prevent hallucination?
- What happens if SYNAPT does not contain the required evidence?
- What happens if two documents conflict?
- What does “not confident” mean?
- When do you escalate?
- How do you determine evidence sufficiency?
- How do you know firmware caused the outage?
- How do you distinguish correlation from causation?
- How do you explain the result?
- How do you observe the agent?
- How do you audit the agent?
- What happens when an SOP does not exist?
- What happens when policy blocks an action?
- Why is human approval required?
- Can the agent directly modify infrastructure?
- How does feedback work?
- How do you handle stale data?
- How do you handle duplicate data?
- How do you handle temporal information?
- How do you handle vendor changes?
- How do you identify similar assets?
- What are the limitations?

Provide technically defensible answers.

===== 31. ACCURACY RULE — VERY IMPORTANT
=====

The SYNAPT training material is the primary source for SYNAPT-specific terminology and behavior.

Do NOT invent proprietary SYNAPT internals.

If something is not explicitly established by the training material, label it:

"Proposed application-level design"

or

"Implementation assumption"

or

"Requires confirmation from the SYNAPT team."

Never present an assumption as an official SYNAPT capability.

===== 32. TEACH ME IN PARALLEL
=====

I am not only looking for the final project.

I need to become confident enough to explain it myself.

Therefore, for every major section:

1. Explain it in very simple language.
2. Explain it technically.
3. Explain WHY it exists.
4. Explain HOW it works.
5. Give a concrete example.
6. Explain the data involved.
7. Explain the expected agent behavior.
8. Give likely SYNAPT questions.
9. Give strong answers.
10. Point out common mistakes I must avoid.

Do not assume I already understand the architecture.

Build my understanding step-by-step.

===== 33. FINAL DELIVERABLE
=====

At the end provide:

- A. Executive Summary
- B. Business Problem
- C. Current vs Proposed Architecture

D. Detailed Data Model

E. Source Systems

F. Adapter Design

G. Ingestion Flow

H. Knowledge Store Design

I. Procedural Store Design

J. Graph Schema

K. Retrieval Strategy

L. MCP Tool Mapping

M. Agent Architecture

N. Agent Registry / Governance

O. Policy Model

P. Human-in-the-Loop

Q. Explainability Design

R. Observability Design

S. Audit / Trace Design

T. Insufficient-Evidence / Escalation Design

U. Feedback Loop

V. Security

W. Complete Architecture Diagram

X. Sequence Diagram

Y. Realistic Demo Dataset

Z. End-to-End Demo

AA. Presentation Slides

AB. Presentation Speaking Script

AC. Technical Q&A

AD. Cheat Sheet

AE. Common Mistakes

===== MOST IMPORTANT REQUIREMENT
=====

I need to understand the complete journey of ONE incident.

Start with:

"Router R-104 has experienced an outage."

Then explain exactly what happens:

What data already exists?

Where did that data come from?

How was it ingested?

Which adapter handled it?

What was stored?

Which entities became graph nodes?

Which relationships were created?

How does the agent receive the incident?

How does Agent Registry govern the agent?

How does the agent decide which SYNAPT tool to use?

What does Trident Search return?

Why does the agent traverse the graph?

What does Trident Traverse return?

When does it use Get Chunks?

When does it use Get Procedures?

How does it retrieve historical RCA?

How does it evaluate policy?

How does it determine whether evidence is sufficient?

How does it construct an evidence-supported root-cause hypothesis?

How does it explain WHY?

How is the agent's execution observed?

How is everything audited?

When is human approval required?

What happens if evidence is insufficient?

What message does the user receive?

When does escalation occur?

How is the final outcome recorded?

How is feedback sent to SYNAPT?

I should be able to explain this entire journey confidently without memorizing meaningless terminology.

The final solution must be:

- Technically realistic
- SYNAPT-aligned
- Governed
- Explainable
- Observable
- Auditable
- Safe
- Demonstrable
- Defensible during Q&A

Most importantly:

NO SUPPORTED EVIDENCE → NO FABRICATED ANSWER → "I AM NOT CONFIDENT" → ESCALATE TO THE APPROPRIATE HUMAN WHEN REQUIRED.

Do not hide uncertainty.

Do not invent SYNAPT capabilities.

Do not turn the project into a generic chatbot.

Build the solution as a realistic enterprise AI-agent architecture using SYNAPT.